FinTech Project 2026

FINS5545: Financial Market Data Literacy

Systematic Multi-Asset Funds with News-Sentiment Analytics

Overview

You will design and build your own prototype FinTech product: an investment app that offers a user several 'systematically managed' funds (a portfolio of assets). You will give your app a name (for example, 'UNSWTrader'). The idea is that an investor can open the app/website, compare your offering of investable funds, read each fund's fact sheet, and decide how to allocate money across them. You build the funds from structured market data as optimal portfolios - equity-only, crypto-only, and combined together - using methods such as maximum-Sharpe (mean-variance tangency), minimum-variance, and risk parity, and you evaluate each with an out-of-sample backtest, so the app shows out-of-sample performance rather than an in-sample fit.

The investable funds are the product: an investor sends money to the investment app and invests in your systematically managed funds, and the business (your app) earns a management fee. Each fund's fact sheet helps the investor choose which fund to invest in - it reports growth of $1 (cumulative return), drawdowns, the Sharpe ratio, average return, and risk, plus the fund's current holdings. The app is a dashboard built so a user can compare funds, read a fact sheet, set an allocation, and (hopefully) invest money in the funds your app offers. This project gives you hands-on experience in building a data-driven investment product from raw data up until the product phase - covering all four stages of the Data Factory Floor (DFF).

Alongside the optimal portfolios, you will also build a news-sentiment index from unstructured news headlines - a standalone analytic across the equity sectors that a user can access over time as part of the app. Innovation is rewarded everywhere in the project, not in one place - a wider set of optimisation methods or funds, a new way to use the news data, a custom figure and design system rather than the provided style, or any app feature you can argue is genuinely valuable. The bar for a distinction grade is work that goes beyond what a simple AI prompt would produce. The grading team have run hundreds of AI-generated projects through our systems - we know exactly what is produced from using a prompt. Top grades are awarded for originality, innovation, and going beyond what a simple prompt would produce for you.

You run the full Data Factory Floor (DFF) end to end and deliver two written reports plus the deployed app (via Streamlit). This aim runs through the whole project: turn raw market and news data into systematic funds, backed by out-of-sample backtested evidence, that an investor could act on and invest in via your product.

Innovation is highly rewarded - a 'prompt and paste' project will not do well.

Innovation & Data-Driven Results is the most heavily weighted criterion in both Parts (25% of Part A and 30% of Part B). Originality also lifts your marks for presentation and design, for economic interpretation, and for your AI workflow. The reason is simple: AI tools can produce a competent baseline on their own, so top grades require going beyond what AI will produce for you. A project that does only what a short AI prompt would produce - clean the data, run a standard backtest, deploy the template app - can still pass, but it will not reach

the higher achievement grade bands. The route to a strong mark is to extend the course baseline in a way you can motivate and show with evidence.

What counts as innovation (broad and illustrative, not a checklist):

- A wider or newer set of funds or optimisation methods than the required minimum.
- A new investment factor or signal.
- A new use of the news data - a custom sentiment tool, an extended finance lexicon, topic or entity tagging.
- A custom figure and design system of your own rather than the provided style, or an original, well-argued app feature.
- A new evaluation method, data-quality check, or robustness test that goes beyond the baseline requirements.

These are examples, not requirements. One well-executed extension is worth more than several shallow ones, and you do not need to cover every category. An extension that you build and evaluate carefully but that does not beat the baseline portfolio return still earns innovation credit: the marks are for original, evidenced work, not for a winning result or funds that produce enormous returns.

Task	Individual project. Two written reports + code + a deployed Streamlit app.
Part A	Data Foundation (Stations 1-2). 20% of course. Due Friday, Week 8.
Part B	Funds, Sentiment & App (Stations 3-4). 50% of course. Due Friday, Week 11.
Where you work	Build everything inside the fins-agent/fins2026/ folder, in your <zID>_projectA or <zID>_projectB subfolder.
Hand-in	One folder per Part, named <zID>_projectA and <zID>_projectB, zipped to Moodle (see Section 6). Part B also: a public GitHub repo + a live Streamlit URL.
Submission	The zipped project folder including report (PDF), code, and your AI workflow pack (your agent files + prompt logs).
Data	Provided online (equity, crypto, news headlines). No scraping or API keys required.
AI policy	AI permitted for the whole project. Your written analysis and economic interpretation must be your own.

1. Goals & Learning Outcomes

Working individually, you run the full Data Factory Floor (DFF) to take raw structured and unstructured data through to a deployed investment app that offers users systematically managed funds to invest in.

Course Learning Outcomes assessed:

- Apply the four-stage Data Factory Floor to a real structured + unstructured data problem.
- Build and critically evaluate out-of-sample portfolios and a news-sentiment model.
- Combine structured and unstructured data into investment decisions and judge their value.
- Communicate financial, data-driven analysis and economic interpretation clearly to a non-technical client.
- Plan and deliver, independently, a professional, reproducible, deployed data product.

Innovation is rewarded across every part of the project and is a distinct, heavily weighted band in both Parts. Students who go beyond a baseline an AI prompt would produce - a richer set of funds, a new use of the news data (for example extending VADER's lexicon via your AI agent), a custom design system, or any feature they can argue is valuable - are highly rewarded.

How the deliverables map to these outcomes:

- **Out-of-sample funds and fact sheets -** delivering systematically managed portfolios an app user can invest in (the core of the product).
- **Sentiment index -** a standalone news analytic across the equity sectors.
- **Innovation -** going beyond a baseline an AI prompt would produce, rewarded across the whole project.
- **Streamlit app and the two reports -** communicating to a non-technical client and delivering a reproducible product.
- **AI workflow pack -** how you direct and audit AI, a fundamental skill this course builds.

2. The Data Factory Floor (DFF)

The project is organised as four Stations, one per DFF stage. Part A covers Stages 1 and 2 (the data foundation). Part B covers Stages 3 and 4 (the models and the product).

- **Station 1 -** Data Lake (ETL): load, clean, and quality-check the structured and unstructured data. Critical attribute: clean data (value + provenance).
- **Station 2 -** Feature Engineering: returns and risk features from prices, and assembling the headlines into a daily text panel (the sentiment model is built in Station 3, not here).
- **Station 3 -** Model Design: the out-of-sample portfolio optimisation (equity + crypto), the sentiment model (VADER or another) that scores the headlines into a sentiment index, and the fusion of the two.
- **Station 4 -** Implementation: a deployed Streamlit application that delivers the product to a client.

3. Data Provided

All data is hosted online as a single ZIP of Parquet files. A provided helper downloads and unzips it once (cached) - no scraping, no API keys, no large downloads. The bundle contains:

- **equity_prices -** 50 US large-cap stocks across 10 sectors, daily OHLCV + adjusted close, 2020-2023.
- **crypto_prices -** 10 major cryptocurrencies, daily OHLCV + adjusted close, 2020-2023 (trades 7 days/week, price-only, no sector).
- **news_headlines -** daily news HEADLINES for the 50 equities (date, ticker, sector, title, url, publisher), 2020-2023.

The 50 equities carry both prices and news, so any work with the sentiment scores - including any sentiment-based extension - applies only to the equity data. Crypto is price-only, a diversifier in the funds. A starter data-access helper is provided. See Appendix A for the data dictionary and load URLs.

4. Part A - Data Foundation (Stations 1-2) - 20%, due Friday Week 8

Part A builds and documents the clean data foundation your funds will run on, and sets out the product. Stations 1 and 2 are the back-end of the product, which the app user never sees, so Part A is where you explore and document the data thoroughly - the news text as much as the prices. The report must stand on its own evidence: every claim is backed by a table or figure, and every table or figure is referenced and interpreted in the text. Write it for a financially literate but non-technical reader - enough technical detail to reproduce your work, but interpret every exhibit in plain English.

Part A at a glance:

- Do, in order: load via data_access, clean and deduplicate, compute returns, left-merge crypto onto the equity calendar, assemble the daily text panel, produce the required exhibits, draft the report, run scripts/check_handin.py.
- Do NOT do Part B work here: no scoring sentiment, no optimising portfolios, no backtesting funds.
- Folder: src/ (code), scripts/ (runnable), results/ (outputs), report/ (writing), ai/ (AI logs).

Common mistakes that lose marks:

- Scoring sentiment in Part A (that is Part B).
- Deduping news on ticker-date alone - use ticker, date, and title.
- Computing crypto returns after merging to equity dates (compute first, then merge).
- Deleting real outliers instead of documenting and keeping them.
- Submitting AI-written prose you have not rewritten in your own words.

Station 1 - Data Collection & Processing

- Name your app and state its value proposition: who the user is, what gap in the market the app fills, and exactly which structured and unstructured inputs it needs.
- Load the hosted data through the provided data-access helper. Document your cleaning and integrity checks: a missing-date audit, a duplicate check, and an outlier or extreme-value screen on returns. The duplicate check differs by dataset - prices should be unique by ticker-date, but news has many rows per ticker-date, so check for exact duplicate headlines on ticker, date, and title. Resolving an outlier can mean verifying and justifying it: the genuine extreme returns here are real events, so keep and explain them rather than delete them. For Part A it is enough to confirm the price and volume fields are internally consistent and state that you keep the observation - external news verification is optional.
- Calendars and merging: equities trade about 252 days a year, crypto about 365. Compute returns within each panel first, then left-merge the crypto returns onto the equity trading calendar to build the combined panel (this intentionally excludes weekend-only crypto moves, which a fund trading on equity days could not act on). The news dates are timezone-aware (UTC) while the price dates are not, so normalise timezone and dtype before any merge. Do not merge price levels across the two calendars and difference afterwards - that creates spurious returns.
- Define your clean output datasets (schema, frequency, coverage) and record provenance (source and how each field was produced).

Station 2 - Feature Engineering & Text Assembly

- Returns: daily returns are the one feature the portfolio optimisation requires. You may also compute rolling volatility and Sharpe and any factors (with equations) to evaluate individual assets, but these are optional and not inputs to the optimiser.
- Descriptive statistics: summarise returns by asset class (mean, volatility, min, max, and a shape measure such as skew or kurtosis).
- Text assembly: structure the headlines into a daily panel per ticker and sector. Map every headline to its equity trading day - the same day if it is a trading day, otherwise the next trading day. Keep the raw headline text - if you plan to score with VADER, do not strip stopwords or 'non-sentiment' words, because VADER relies on them. The sentiment model itself - scoring the text, and lagging the signal to avoid look-ahead - is built in Part B, Station 3, not here.
- Explain, for each feature, why it is useful, how it is computed, and what you expect it to look like.

Required exhibits (Part A) - each self-contained (caption, labelled axes, units, sample period) and interpreted in the text:

- A dataset-inventory table: rows, date span, frequency, and coverage for equities, crypto, and headlines.
- A data-integrity summary: missing dates, duplicates, and outliers found and how you resolved them.
- A descriptive-statistics table of returns by asset class.
- A price or cumulative-return figure for a sample of equities and crypto.
- A returns-distribution or outlier figure (for example a histogram or boxplot).
- A descriptive analysis of the text data: the number of articles over time and a count of sentiment-bearing words (counting vocabulary is descriptive and belongs in Part A - scoring and indexing sentiment is Part B), plus any other counts that describe the news flow.
- A text-exploration figure - for example a word cloud of the most frequent words, or a bar chart of the top terms.

Required output filenames (use these exact names so markers can find them): save the dataset-inventory table as results/tables/dataset_inventory.csv and the returns descriptive-statistics table as results/tables/descriptive_stats_returns.csv. Other exhibits (for example results/figures/returns_distribution.png) can use any clear name.

Suggested Part A report structure (max 2,500 words, about 5 pages, excl. appendix and references - you may place the required exhibits in an appendix): (1) product and value proposition, (2) data sources and cleaning, (3) integrity checks and descriptive statistics, (4) return features and the text data exploration, (5) what the foundation enables next. Author the report in Word (report/report.docx is the editable source, and OUTLINE.md is only a planning aid) and submit it as report/report.pdf. Deliverable: this report, your code and a small sample of derived data under results/data/ (for example combined_returns_panel.csv - derived artifacts only, never raw .parquet or source data), and your AI workflow pack (your own agent/instruction files and prompt logs).

Hand-in: zip the whole folder, named <zID>_projectA (for example z3539841_projectA), and upload the zip to Moodle. See Section 6 for the folder layout.

5. Part B - Funds, Sentiment & App (Stations 3-4) - 50%, due Friday Week 11

Part B builds the funds, the sentiment analytics, and the app. The report focuses on model output and on how the app works. As in Part A, every exhibit is self-contained and interpreted.

Part B at a glance:

- Required minimum: a combined equity-plus-crypto fund with at least two optimisation methods. Higher band: equity-only and crypto-only funds, extra or novel methods, a sentiment fusion that adds value, and standout app features.
- Build it: reuse your own Part A foundation, then build the out-of-sample funds and fact sheets, the sentiment model and sector index, and the Streamlit app.
- Artifacts: write app-readable CSVs to results/data/ (committed, the app reads them), report tables to results/tables/, and figures to results/figures/. Raw .parquet and source data are never committed.
- Run order: python scripts/run_part_b.py, then streamlit run streamlit_app.py, then python scripts/check_handin.py, then git status.

Common mistakes that lose marks:

- Making the deployed app recompute backtests or run VADER - load precomputed results/ instead.

- Submitting a private repo at hand-in.

Station 3 - Funds, Backtests & Sentiment

- Funds (required minimum): build a combined equity-plus-crypto fund with at least two optimisation methods (for example maximum-Sharpe / mean-variance tangency, minimum-variance, risk parity, or equal-weight). Treat each (asset family, method) pair as one fund - for example 'Combined Minimum-Variance' - since that is what a user invests in and what a fact sheet covers. Equity-only and crypto-only funds, extra optimisation methods, and newer or more advanced methods you devise are exactly the kind of work that earns innovation marks.
- Backtest rules - a walk-forward out-of-sample backtest with no look-ahead bias, weights formed only from past data, and rebalancing monthly or less often (for example the first or last trading day of each month, every 21 trading days, or quarterly). The out-of-sample period starts after the initial estimation window, not on the first date in the data - state your first live backtest date and the window length. You may assume a risk-free rate of zero for the Sharpe ratio (state your choice), or download a proxy of the risk-free rate as part of your analysis. Zero transaction costs are acceptable if stated, and adding a turnover or transaction-cost model counts as an innovation. Choose your own window type, rebalance frequency, and constraints.
- Fact sheet per fund (one per (family, method) fund): growth of $1 (cumulative return), annualised return, annualised volatility, Sharpe ratio, maximum drawdown, and current holdings (the target weights from your most recent rebalance). Compare the funds.
- Sentiment model: apply a sentiment model (VADER or another) to score the assembled headlines, then build a standalone news-sentiment index across the equity sectors and show it over time. Build the sector index by averaging ticker-day sentiment within each sector (equal-weight the tickers), and decide how to treat ticker-days with no headlines (drop, carry forward, or treat as neutral) and justify it. Lag the sentiment signal by at least one trading day relative to the trading day it is aligned to, so day t's decision uses only sentiment from day t-1 or earlier (a Saturday or Monday headline, both aligned to Monday, is first usable for Tuesday's trade). Justify your text-handling choices (for example why you do or do not strip casing and punctuation).
- Combining structured and unstructured data (expected - a basic attempt is fine): fold the equity sentiment into the equity funds (for example a sentiment tilt or factor), and report its before-vs-after effect. A naive attempt that underperforms is fine - this is a baseline, and going beyond it (a better fusion, a tuned tilt, a sentiment factor) is where the innovation marks can be earned. Any sentiment work applies to the equity data only.

Innovation is rewarded across the whole project, not only in the sentiment extension. Examples: a wider array of funds and optimisation methods, extending VADER's lexicon (for example, having your AI agent propose finance terms and assign them sentiment scores), a custom figure and design system rather than the provided style, or any app feature you can argue is genuinely valuable.

Required exhibits (Part B) - each self-contained and interpreted:

- A performance-metrics table across funds and methods (annualised return, volatility, Sharpe, max drawdown).
- A growth-of-$1 (cumulative-return) figure comparing the methods.
- A drawdown figure for at least one fund.
- A portfolio-weights-over-time figure across methods for at least one fund.
- A Sharpe (or return-vs-risk) barplot across funds and methods.

- A sentiment-index time series for the equity sectors.
- A fusion before-vs-after comparison (base fund vs sentiment-augmented), as a table and a figure.

Required output filenames (the app reads them and markers check them, so use these exact names): results/data/fund_returns.csv, results/data/fund_weights.csv, results/data/sector_sentiment_index.csv, and results/tables/performance_metrics.csv. Any additional artifacts can use any clear name.

Station 4 - Implementation

- Build a Streamlit app for the investor journey: compare the funds, open a fund's fact sheet, set an allocation across funds, and read the sentiment analytics.
- Deployment: your AI agent can prepare the repo, run scripts/check_handin.py, and push it, but the final deploy is browser-based and needs your own GitHub and Streamlit login, so you finish that step. The hand-in folder is the repo root, so the app entrypoint is streamlit_app.py. Deploy from a PUBLIC GitHub repo at hand-in (private while you build). Submit the live URL and the public repo link. See Appendix D.
- Load raw data through the data-access helper (never commit raw data). Commit your precomputed app artifacts under results/ - the deployed app reads them and must not import nltk or recompute backtests, since the free tier cannot. The full build may take a few minutes locally, but the deployed app loads fast because it only reads precomputed artifacts. Keep the app light so it runs on a basic machine.
- Describe the target user and the customer journey in the report.

Suggested Part B report structure (max 10 pages of written narrative, about 5,000 words, excl. appendix and references - exhibits may go in an appendix): (1) the funds and the backtest design, (2) out-of-sample results and fund fact sheets, (3) the sentiment index, (4) your extensions and innovations, (5) the app and the investor journey, (6) critical reflection with three concrete recommendations. Author the report in Word (report/report.docx is the editable source, and Markdown drafts are fine as planning aids) and submit it as report/report.pdf. Deliverable: this report, full code, the live app URL and public repo, and your AI workflow pack.

6. What You Hand In

Each Part is one folder named with your zID: <zID>_projectA for Part A (for example z3539841_projectA) and <zID>_projectB for Part B. Download the starter folder for the Part from Moodle (projectA_starter.zip or projectB_starter.zip), unzip it into fins-agent/fins2026/, and rename the folder to your zID name. You do ALL your work inside that one folder: it is what you run on your computer, what you zip and upload to Moodle, and - for Part B - what you deploy. Whatever happens with GitHub or the live app, that folder on your computer keeps working, so you can always run it and submit it. Every student starts from the same structure so nothing is lost or misplaced:

- **PROJECT_BRIEF.md -** this brief, included in the folder so your AI agent can read it.
- **README.md -** how to run your code and what you built.
- **report/ -** your written report (PDF).
- **src/ -** your code, including the provided data-access helper.
- **scripts/ -** runnable scripts that reproduce your results.
- **results/ -** the figures and tables your code produces.
- **context/ -** the provided data guide and project context (do not edit).
- **ai/ -** your prompt logs and AI-use notes.
- **AGENTS.md / CLAUDE.md -** your own AI agent instruction files (see Section 7).

- **SUBMISSION_CHECKLIST.md -** tick every item before you hand in.

For Part B the same folder also holds the app at its root (streamlit_app.py, .streamlit/, requirements.txt, requirements-dev.txt) and becomes your own GitHub repository, with the app entrypoint streamlit_app.py at the folder root. Your AI assistant turns this one folder into a GitHub project and uploads it (kept private while you build); you finish by clicking Deploy in the browser, and make the repository public at hand-in. Submit the live Streamlit URL and the public repository link alongside the zip. See Appendix D for the steps.

7. Use of AI

- AI tools are permitted for the entire project - coding, debugging, design, and drafting.
- Your AI workflow is GRADED: 20% of each Part (the AI Workflow & Transparency criterion).
- Your agent files are your own work. Replace the placeholder AGENTS.md (Codex), CLAUDE.md and .claude (Claude Code), GEMINI.md (Gemini), or the equivalent for your assistant, with your real instructions, and keep your prompt logs in ai/. They live at the project-folder root (including a project-local .claude/ if you use one) and are submitted with the zip. Do not submit the placeholder unchanged.
- Open only your own project folder in your AI tool (your own Part A and Part B) - not the whole fins-agent repository or anyone else's folder - so the assistant only ever sees your own work.
- Acceptable: ask AI to draft a backtest function, then read it, test it, fix the look-ahead bug it introduced, and record the prompt, the output, and your fix with the reason.
- Not acceptable: paste AI-generated portfolio code or written analysis into your submission unread and claim it as your own.
- A good AI-log entry (Part A): the prompt you used to generate the ETL code, what the AI got wrong (for example it merged before differencing), how you checked the calendar alignment, and what you changed.
- Your written analysis and economic interpretation must be your own. AI-generated reasoning submitted as your own is penalised.
- Because AI makes execution easy, innovation, interpretation, and a well-documented AI process are where marks are won.

8. Important Points (read before you start)

- Where you work: do everything inside your fins-agent/fins2026/<zID>_projectA or <zID>_projectB folder. That one folder is your runnable code, your Moodle zip, and (Part B) your deployed app, and it keeps working on your computer no matter what happens online.
- Exhibits are evidence: every figure and table must be self-contained (caption, labelled axes, units, sample period) and be referenced and interpreted in the report - never drop in a raw plot.
- Calendar mismatch: equities trade about 252 days/year, crypto about 365. Align calendars before combining, and annualise with the right factor.
- Headlines, not articles: the news data is headlines only. Headline sentiment is a noisy proxy - say so.
- VADER neutrality: about half of finance headlines score neutral with plain VADER, and many are false neutrals. A finance lexicon helps, and a sentiment of zero is not 'no information'.
- Adding sentiment does not guarantee better returns: a naive sentiment tilt can underperform the base portfolio.
- Solver scaling: optimisers on tiny daily-return covariances can silently stall (objective below tolerance). Sanity-check that weights actually change across methods.

- Data is one ZIP: the helper downloads + unzips it once and caches it. Do not re-download on every interaction, and keep the app light.
- Public repo at hand-in: make the GitHub repo public and the app live before the deadline. Private-repo apps will not run for markers.
- check_handin.py: the starter intentionally fails until you replace at least one agent file (AGENTS.md or CLAUDE.md) with your own. Fix every [FAIL] - the [WARN]s are only reminders.

9. Marking Rubric

UNSW HD/D/C/P/F bands. Weights are shares of each Part. Innovation & Data-Driven Results is the most heavily weighted band in both Parts (25% of Part A, 30% of Part B), and originality also lifts the presentation and design, interpretation, and AI-workflow marks.

Part A criteria (20% of course):

Criterion	Wt	HD (85-100)	D (75-84)	C (65-74)	P (50-64)	F (<50)
Data Collection & ETL (Station 1)	**15%**	Both price datasets and the news headlines are loaded through the data-access helper and summarised in a dataset-inventory table (rows, date span, frequency, coverage). Integrity checks are run and documented - a missing-date audit, a duplicate ticker-date check, and an outlier or extreme-value screen - and the equity (252-day) and crypto (365-day) calendars are aligned. Issues found are quantified and resolved, not just listed, and the clean schema and data provenance are stated.	Both data types are loaded with an inventory table and most integrity checks (missing dates, duplicates, outliers) documented. Calendar handling is correct, with small gaps in provenance or in resolving the issues found.	Data loaded with basic cleaning and one or two integrity checks. The calendar mismatch is acknowledged and documentation is partial.	Data loaded, but checks are minimal, partly incorrect, or undocumented, or the calendar mismatch is ignored.	Data not loaded correctly, with no meaningful cleaning, checks, or schema.
Feature Engineering & Text Assembly (Station 2)	**15%**	Returns are computed correctly (the one feature the optimisation needs) and summarised in a descriptive-statistics table, with optional risk features or factors defined by equations. The headlines are assembled into a daily text panel per ticker and sector, date-aligned to the trading calendar, ready for the Station 3 sentiment model. Each feature's purpose	Sound return features with a statistics table, and a working daily text panel assembled from the headlines. Minor gaps in justification.	Adequate return features and a basic text panel, but with limited justification or a missing statistics table.	Basic features. Returns are mis-computed (for example across the merge) or the text panel is incomplete or mis-aligned.	Features or text assembly missing or fundamentally flawed.

		and expected behaviour is stated.				
Innovation & Data-Driven Results	**25%**	One or more original extensions that go beyond the course baseline and beyond what a simple AI prompt produces, implemented and shown with evidence rather than only proposed. Any one suffices: a new feature or investment factor defined by its equation and tested, a wider or novel use of the news data, a custom data-quality or analysis method, an original figure or design system, or a new product idea built on the data. The extension is motivated, built on the data, and its outcome is reported and interpreted. A careful extension that does not beat the baseline, clearly explained, still earns this band - the credit is for original, evidenced work, not for outperformance.	At least one genuine extension beyond the baseline, implemented and motivated, shown with evidence, with minor gaps.	A modest extension that is partly original but close to the standard course approach, or proposed more than shown.	Little originality - the work follows the provided examples and the AI-baseline output with no real extension.	No evidence of original, data-driven work beyond the baseline.
Economic Interpretation & Writing	**10%**	Correct economic reasoning, in the student's own words, that explains why each result looks as it does. Writing is precise and well-structured, and every figure and table is referenced and interpreted in the text, not dropped in raw.	Good interpretation tied to the evidence and clear writing, with minor lapses.	Adequate interpretation, with some claims unclear, unsupported, or merely descriptive.	Weak, largely descriptive writing that restates outputs without explaining them.	Incorrect or absent interpretation, and poor writing.
Presentation, Design & Reproducibility	**15%**	Figures and tables are self-contained (caption, labelled axes, units, sample period) and the work is fully reproducible - the code runs end-to-end from the hosted data on a clean checkout of the standard project folder. The top band also shows distinctive visual quality: either an original, coherent design	Clean, self-contained, well-labelled exhibits using the provided style competently, and code that runs with minor friction.	Acceptable presentation, with reproducibility partial (some manual steps or missing labels).	Cluttered or unlabelled exhibits, and code is hard to run.	Poor presentation, not reproducible.

		system (the student's own colour, type, and figure language) or clarity that goes beyond the provided style. Using the provided style well, cleanly, reaches the band below.				
AI Workflow & Transparency	**20%**	The student's own agent or instruction files (AGENTS.md, CLAUDE.md, .claude, or the equivalent for their tool) plus curated prompt logs that show the prompts used, the AI outputs, and the student's own corrections with reasons. A candid, reflective account of where AI helped, where it was wrong, and what the student did instead.	Own agent or instruction file(s) plus prompt logs with some critical evaluation of AI outputs.	A basic AI log: prompts and a short description of use, with limited evaluation or correction.	Minimal AI documentation, with prompts sparse or undescribed, or no agent file.	No AI-process documentation, or undisclosed or deceptive AI use.

Part B criteria (50% of course):

Criterion	Wt	HD (85-100)	D (75-84)	C (65-74)	P (50-64)	F (<50)
Funds: Optimal Portfolios & OOS Backtest (Station 3)	**15%**	Equity-only, crypto-only and combined funds across several optimisation methods, each with a correct walk-forward out-of-sample backtest (no look-ahead, weights from past data only, correct 252 vs 365 annualisation). Fund fact sheets and the required exhibits (a metrics table across funds and methods, growth of one dollar, drawdown, and portfolio weights over time) are present, and the funds are compared.	The combined fund plus at least one single-asset fund across several methods, with a correct out-of-sample backtest, fact sheets, and the core exhibits, with minor gaps.	At least the required combined fund with two methods, backtested out-of-sample with a basic fact sheet. Single-asset funds, extra methods, or some exhibits are missing.	Portfolios formed but below the required minimum, or with look-ahead, annualisation, or calendar errors, or with no fact sheet or exhibits.	No working fund or backtest, or major methodological flaws.
Sentiment Index (standalone) & Fusion Extension (Station 3)	**10%**	A sentiment model (VADER or another) applied to the headlines to build a validated standalone sentiment index across the equity sectors, shown over time, plus a look-ahead-safe	A solid sector sentiment index plus a working, look-ahead-safe fusion attempt, mostly sound.	A sentiment index built and shown, with a fusion attempt that is shallow or weakly evaluated.	Sentiment computed but the index is weak or unvalidated, or the fusion is look-ahead-unsafe.	No working sentiment index.

		fusion of sentiment into the equity funds whose effect is measured and critically assessed. A negative result, explained, counts as strong work.				
Innovation & Data-Driven Results	**30%**	A distinctive, implemented extension shown to advance on the baseline with evidence. Any one suffices: a novel investment factor, a wider or newer optimisation or fund design, a new use of the news data, a custom sentiment tool or lexicon, an original evaluation method, a custom figure and design system, or a genuinely valuable app feature. An original contribution that is built and demonstrated, not just proposed. A careful extension with a negative result, explained, still earns this band - the credit is for evidenced original work, not for outperformance.	A clear original extension beyond the baseline, implemented and motivated, shown with evidence.	A modest, partly original extension, or one proposed more than shown.	Minimal originality - mostly baseline replication and AI-prompt output.	No original contribution.
Streamlit App & Implementation (Station 4)	**15%**	A reliable Streamlit app, deployed from a public GitHub repo, that loads the hosted data and supports the full investor journey (compare funds, read each fund's fact sheet, set an allocation) and surfaces the sentiment analytics, running on a basic machine. Polished, coherent design and user experience - including an original design system - strengthens this band.	A working deployed app with a clear user journey and good responsiveness, with minor issues.	An app that runs and deploys but is basic, partly unreliable, or covers only part of the investor journey.	App incomplete or not reliably deployed (for example a private repo at hand-in, or errors on load).	No working or deployed app.
Economic Interpretation, Critical Reflection & Writing	**10%**	Evidence-based reflection on what worked, what did not, and why, with three concrete and specific real-world recommendations. Clear writing in the student's own words, with every exhibit interpreted.	Solid reflection and specific recommendations, with good writing.	Reasonable reflection, with generic recommendations.	Shallow reflection, with weak or largely descriptive writing.	No meaningful reflection.
AI Workflow &	**20%**	Across the whole	Own agent or	A basic AI log:	Minimal AI	No AI-process

Transparency		build, the student's own agent or instruction files (AGENTS.md, CLAUDE.md, .claude, or the equivalent for their tool) plus curated prompt logs showing the prompts, the AI outputs, and the student's own corrections with reasons. A candid, reflective account of where AI helped, where it was wrong, and what the student did instead.	instruction file(s) plus prompt logs with some critical evaluation of AI outputs.	prompts and a short description of use, with limited evaluation or correction.	documentation, with prompts sparse or undescribed, or no agent file.	documentation, or undisclosed or deceptive AI use.

Mandatory Requirements:

- **Mandatory AI submission -** AI workflow is graded (20% of each Part). You MUST hand in the agent or instruction files you actually used (AGENTS.md, CLAUDE.md, .claude/*, GEMINI.md, or your tool's equivalent) and your prompt logs. These are YOUR files: the provided stubs must be replaced with your own. No submission of these caps the AI Workflow criterion at F.
- **Own writing & interpretation -** AI may assist execution, but the written analysis and economic interpretation must be your own. Verbatim AI prose presented as your reasoning is penalised.
- **Academic integrity -** Standard UNSW academic-integrity rules apply, including correct citation of all data sources and methods. Use only your own work - you may reuse your own Part A in Part B, but do not read or copy from another student's project folder. Undisclosed or deceptive AI use is misconduct.

Appendix A - Data Dictionary & Access

All project data is one ZIP of three Parquet files, hosted here: the official data ZIP (Google Drive). The provided data-access helper (src/data_access.py) downloads the ZIP, unzips it in memory, and caches it, so your code and the app never manage files or hit the network again. Just call its load functions (load_equity_prices, load_crypto_prices, load_news_headlines). To work offline, set the FINS_DATA_ZIP environment variable to a local copy of the ZIP. If the primary source is down, the helper falls back automatically to a backup copy. The three files:

- **equity_prices -** [ticker, date, open, high, low, close, adjClose, volume, sector]. 50 US stocks across 10 sectors, daily, 2020-2023 (50,300 rows).
- **crypto_prices -** [ticker, date, open, high, low, close, adjClose, volume]. 10 cryptocurrencies (e.g. BTC-USD, ETH-USD - all carry a -USD suffix), daily, 2020-2023 (14,620 rows). No sector column - crypto is price-only. 10 rows are dated 2024-01-01, so cap your sample at 2023-12-31.
- **news_headlines -** [date, ticker, sector, title, url, publisher]. 149,683 rows for the 50 stocks, 2020-2023, before de-duplication (about 2,847 are exact duplicates on ticker+date+title). The date is UTC and timezone-aware while the price dates are not, and publisher is often blank.

Appendix B - Technical Requirements

Python with pandas, numpy, scipy, nltk (VADER), matplotlib/plotly, pyarrow, streamlit. VADER needs a one-time nltk.download('vader_lexicon') before it scores. The app's requirements.txt is kept slim - build and

reproduction-only packages (nltk) live in requirements-dev.txt so the deployed app stays light. Everything runs on a basic laptop and on Streamlit Community Cloud.

Appendix C - The Data Factory Floor

See the course textbook, Chapter 1, for the four-stage DFF and the critical attribute of each stage.

Appendix D - Deploying the App (Part B)

Your AI agent can prepare and push the repo, but the final deploy is browser-based and needs your own GitHub and Streamlit login, so you finish it. In practice the agent can commit, create a private repo, and push if your CLI is authenticated, and you complete the Streamlit Cloud browser step. The steps:

- Your <zID>_projectB folder is its own GitHub repository, independent of fins-agent. The app entrypoint is streamlit_app.py at the folder root.
- Commit your code AND your precomputed app artifacts under results/ (the app reads them, and the free tier cannot recompute heavy backtests). Do not commit raw data or secrets - data loads via the data-access helper.
- Test locally first: streamlit run streamlit_app.py.
- Run scripts/check_handin.py (your agent can do this). It checks the folder name, the entrypoint, requirements, and that no raw data or secrets are committed.
- Your AI assistant turns the folder into its own GitHub repository and uploads its contents to a NEW repository (private while you build). This repository is separate from fins-agent, even though the folder sits inside fins-agent/fins2026/ while you work - your working folder on disk is untouched and keeps running locally.
- YOU then deploy in the browser: on share.streamlit.io, sign in, and create a new app from your repo with entrypoint streamlit_app.py. Streamlit deploys from whichever branch holds the app (main or master) - pick it in the connect dialog. Your AI agent cannot do this step - it needs your login.
- At hand-in, make the repo PUBLIC, confirm the live app still loads, and submit the live URL and the repo link.